



Assignment 2

Course	Programming Fundamentals
Instructor	Ms. Hina Iqbal
Section	BCS-1E, BCS-1F

Important Guidelines:

1. If you use **AI tools** (ChatGPT, Copilot, etc.) or **copy code from other students/online sources**, your assignment will be given **zero marks** without exception.

Prophet Muhammad (SAW) said: “Truthfulness leads to righteousness, and righteousness leads to Paradise” - Sahih Al Bukhari 6094

2. Submit your assignment **on time**. Late submissions will incur a 25%-mark deduction if submitted within 6 hours, a 50%-mark deduction if submitted within 24 hours, and submissions beyond 24 hours will result in a score of zero.
3. No marks will be awarded to individuals who do not submit the file on Google Classroom (GCR).

4. Only submit your cpp files on google classroom. **Do not zip your files.** Each cpp file should be renamed according to the format: **I25xxxx_q1.cpp**. **Failure to follow this format will result in a score of zero.**
 5. In case of confusion please feel free to contact on email I227910@lhr.nu.edu.pk or hina.iqbal@nu.edu.pk
-

◆ Questions

Q1. Cafeteria Order Summary with Validation

A university cafeteria wants to generate a daily sales summary.

- Input the number of students who bought food today.
- For each student, input how many **burgers**, **sandwiches**, and **drinks** they bought.
- Use loops to calculate and display the **total number of burgers, sandwiches, and drinks sold**.
- Additionally, calculate and display which item sold the most.
- If any student enters a **negative number of items**, ask them again until they enter valid values.

Example Input/Output:

```
Enter number of students: 3
Student 1 → Burgers: 1 Sandwiches: 0 Drinks: 2
Student 2 → Burgers: 2 Sandwiches: 1 Drinks: 1
Student 3 → Burgers: -1 Sandwiches: 2 Drinks: 0
Invalid input! Enter again: Burgers: 0 Sandwiches: 2 Drinks: 0

Total Burgers Sold = 3
Total Sandwiches Sold = 3
Total Drinks Sold = 3
Most Sold Item: Burgers and Sandwiches (tie)
```

Q2. Seating Chart with Reserved Seats

The university auditorium has **R rows** and **C columns** of seats.

- Input the values of **R** and **C**.
- Input the number of reserved seats, then input their row and column positions.
- Display the seating arrangement using **nested loops** where:
 - Reserved seats are shown as **[X]**
 - Available seats are shown as **[row,column]**
- Handle edge cases for any invalid inputs

Example Input/Output:

```
Enter rows: 3
Enter columns: 4
Enter reserved seats: 2
Reserved Seat 1 → Row: 1 Col: 2
Reserved Seat 2 → Row: 3 Col: 4

[1,1] [X] [1,3] [1,4]
[2,1] [2,2] [2,3] [2,4]
[3,1] [3,2] [3,3] [X]
```

Q3. Advanced Multiplication Tables

Write a program that:

- Takes a number **n** from the user.
- Prints multiplication tables from **1 to n**.
- Each table should only display results where the **product is even**.
- Use **nested loops** to achieve this.

Example Input/Output:

```
Enter n: 3

Table of 1
1 x 2 = 2
1 x 4 = 4
1 x 6 = 6
1 x 8 = 8
1 x 10 = 10

Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
...

Table of 3
3 x 2 = 6
3 x 4 = 12
3 x 6 = 18
...
```

Q4. Attendance and Performance Report

A class has N students. For each student, input the **number of days present (out of 30)** and their **average marks**.

- Use a loop to process all students.
- Classify students into categories:
 - **Scholar:** Attendance ≥ 27 AND Marks ≥ 80
 - **Regular:** Attendance ≥ 20 AND Marks between 60–79
 - **Weak:** Otherwise
- Display the **total count** of each category.

Example Input/Output:

```
Enter number of students: 4
Student 1 → Attendance: 28 Marks: 85
Student 2 → Attendance: 19 Marks: 75
Student 3 → Attendance: 25 Marks: 70
Student 4 → Attendance: 30 Marks: 95

Scholar: 2
Regular: 1
Weak: 1
```

Q5. Spiral Number Pattern

Write a program that:

- Takes an integer n as input (the size of the square matrix).
- Fills the matrix with numbers from 1 to $n \times n$ in a **spiral order starting from the center**.
- Prints the matrix in proper format.

Example Input/Output:

```
Enter the size of the spiral: 5

17 16 15 14 13
18  5  4  3 12
19  6  1  2 11
20  7  8  9 10
21 22 23 24 25
```

Q6. Exam Results Analyzer with Topper

The department wants to analyze exam results.

- Input the number of students.
- For each student, input **marks of 5 subjects**.
- Use nested loops to calculate each student's **total** and **average marks**.
- At the end:
 - Display the **class average**.
 - Display the **roll number of the student with the highest average marks**.

Example Input/Output:

```
Enter number of students: 2

Student 1 → Marks: 80 70 90 85 75
Total = 400 Average = 80
Student 2 → Marks: 60 65 70 55 50
Total = 300 Average = 60

Class Average = 70
Topper: Student 1 with 80 average
```

Q7. Number Guessing Game with Attempt Limit

Write a loop-based game where:

- The program has a **secret number**
- User keeps guessing until they enter the **correct number** OR until they reach **5 attempts**.
- After each wrong guess, display "Try again!".
- If attempts run out, display "Game Over!".

Example Input/Output:

```
Guess the number: 4
Wrong! Try again.
Guess the number: 9
Wrong! Try again.
Guess the number: 10
Correct! You win.
```
